CLASSIC ASP WEB FOUNDATIONS · ONE-DAY BUILD

Placement Application Tracker

Build a real CRUD web application for tracking job and internship applications. Every hour produces something new that works in the browser.

TIME
About 10 hours

INTERFACE
Browser-based

DATABASE
SQL Server Express

OUTCOME
Complete CRUD app

---

01 · THE BRIEF

Build something useful today

Students apply to several companies and quickly lose track of roles, dates, statuses, and notes. Build a web application that keeps this information in one place. A user must be able to create, read, update, and delete applications entirely through browser pages.

The final demonstration

Open the site in IIS, open the dashboard, add an application, view it, edit its status, delete another record, restart IIS (`iisreset`) or recycle the app pool, and prove the remaining data is still present in SQL Server.

---

02 · TECHNOLOGY

A small Classic ASP stack

Classic ASP (VBScript)
Server-side pages

One `.asp` file per page: read `Request`, write `Response`, redirect, set status codes.

IIS / IIS Express
Web server

Hosts the pages, runs the ASP engine, and keeps the `Session` used for CSRF tokens and flash messages.

ADO
Database access

`ADODB.Connection` and `ADODB.Command` with parameters. SQL is never built by string concatenation.

SQL Server Express
Relational database

A free, real database server with typed columns, constraints, and SSMS for inspecting data.

Includes + HTML/CSS
Browser interface

`<!--#include-->` header and footer for a shared layout; every value printed through `Server.HTMLEncode`.

Checklist + PowerShell
Verification

A manual test checklist, an `Invoke-WebRequest` smoke script, and SQL queries that prove what was saved.

What carries over from this build

Classic ASP is a legacy platform, so the value is in the ideas, not the framework. Everything here transfers directly to ASP.NET, Flask, FastAPI, or Node: routes and HTTP methods, query string and form data, server-side validation, POST/Redirect/GET, status codes, output encoding against XSS, parameterized SQL against injection, CSRF tokens, sessions, and real SQL Server schema design with constraints. The T-SQL schema and queries can be reused unchanged by a newer application.

Prerequisites

Windows and IIS

- Windows Features: **Internet Information Services** → **World Wide Web Services** → **Application Development Features** → **ASP** (plus IIS Management Console).
- Or IIS Express with ASP enabled in `applicationhost.config`.
- Create site `PlacementTracker` pointing to `C:\inetpub\placement_tracker`, port 8080.
- App pool: **No Managed Code**, integrated pipeline. On 64-bit with an old 32-bit provider, set *Enable 32-Bit Applications*.
- Turn on *Send Errors To Browser* for development only (IIS → ASP → Debugging Properties).

SQL Server Express

- Install SQL Server Express (instance `.\SQLEXPRESS`) and SSMS.
- Install the **Microsoft OLE DB Driver for SQL Server** (`MSOLEDBSQL`); `SQLOLEDB` is the built-in fallback.
- Create database `PlacementTracker` and run `db\schema.sql`.
- Permissions: add login `IIS APPPOOL\PlacementTracker`, map it to the database with `db_datareader` and `db_datawriter` only.
- Give the app pool identity read access to the site folder.

---

03 · PRODUCT SPECIFICATION

Pages and behavior

Dashboard · `default.asp`

- Show every application in a table.
- Show company, role, status, and applied date.
- Link to details, editing, and creation.
- Show a friendly empty state when no records exist.

Create application · `new.asp`

- Display a browser form.
- Require company, role, status, and applied date.
- Reject invalid values with useful messages.
- Redirect to the new record after saving.

Application details · `view.asp?id=`

- Display every field for one record.
- Return a proper not-found page (`404`) for an unknown or non-numeric ID.
- Provide edit, delete, and back controls.

Edit and delete · `edit.asp` , `delete.asp`

- Pre-fill the edit form with existing data.
- Persist valid updates to the database.
- Delete through a POST request with confirmation and a CSRF token.
- Never delete data from an ordinary link click.

```
PLACEMENT TRACKER                          [+ Add application]

Applications (3)
```

Company	Role	Status	Applied on	Actions
Acme Labs	Software Intern	Applied	16 Sep	View
Northstar	Graduate Engineer	Interview	12 Sep	View
Contoso	Backend Intern	Rejected	05 Sep	View

04 · TECHNICAL CONTRACT

Data model and routes

Application record (table `dbo.Applications`)

Field	SQL Server type	Rules
`id`	INT IDENTITY	Primary key, generated automatically
`company`	NVARCHAR(100)	Required, 2 to 100 characters
`role`	NVARCHAR(100)	Required, 2 to 100 characters
`status`	NVARCHAR(20)	Wishlist, Applied, Interview, Offer, or Rejected
`applied_on`	DATE NULL	Required unless status is Wishlist
`job_url`	NVARCHAR(500) NULL	Optional; must start with http:// or https://
`notes`	NVARCHAR(1000) NULL	Optional, maximum 1,000 characters
`created_at`	DATETIME2	Generated automatically (UTC)

Schema script · `db/schema.sql`

```
CREATE TABLE dbo.Applications (
  id          INT IDENTITY(1,1) CONSTRAINT PK_Applications PRIMARY KEY,
  company     NVARCHAR(100)  NOT NULL CONSTRAINT CK_App_Company CHECK (LEN(company) BETWEEN 2
  role        NVARCHAR(100)  NOT NULL CONSTRAINT CK_App_Role    CHECK (LEN(role) BETWEEN 2 AN
  status      NVARCHAR(20)   NOT NULL CONSTRAINT CK_App_Status
                CHECK (status IN (N'Wishlist',N'Applied',N'Interview',N'Offer',N'Rejected')),
  applied_on  DATE           NULL,
  job_url     NVARCHAR(500)  NULL,
  notes       NVARCHAR(1000) NULL,
  created_at  DATETIME2(0)   NOT NULL CONSTRAINT DF_App_Created DEFAULT SYSUTCDATETIME(),
  CONSTRAINT CK_App_AppliedOn CHECK (status = N'Wishlist' OR applied_on IS NOT NULL)
);
```

Connection · `inc/db.asp`

```
<%
Const CONN_STR = "Provider=MSOLEDBSQL;Server=.\SQLEXPRESS;Database=PlacementTracker;Integrate
' Fallback: "Provider=SQLOLEDB;Data Source=.\SQLEXPRESS;Initial Catalog=PlacementTracker;Inte

Function OpenDb()
  Dim cn : Set cn = Server.CreateObject("ADODB.Connection")
  cn.Open CONN_STR
  Set OpenDb = cn
End Function
%>
```

Parameterized read · `view.asp` (excerpt)

```
<%
Dim id : id = Request.QueryString("id")
If Not IsNumeric(id) Or id = "" Then NotFound()

Dim cn, cmd, rs
Set cn = OpenDb()
Set cmd = Server.CreateObject("ADODB.Command")
Set cmd.ActiveConnection = cn
cmd.CommandText = "SELECT id, company, role, status, applied_on, job_url, notes, created_at "
                  "FROM dbo.Applications WHERE id = ?"
cmd.Parameters.Append cmd.CreateParameter("id", 3, 1, , CLng(id))   ' adInteger, adParamInput
Set rs = cmd.Execute
If rs.EOF Then NotFound()
%>
<h1><%= Server.HTMLEncode(rs("company")) %></h1>

<%  ' inc/helpers.asp
Sub NotFound()
  Response.Clear
  Response.Status = "404 Not Found"
  Server.Execute "/404.asp"
  Response.End
End Sub
%>
```

Route contract

Method	Page	Purpose	Result
`GET`	`/default.asp`	List applications	Dashboard HTML
`GET`	`/new.asp`	Show create form	Form HTML
`POST`	`/new.asp`	Create a record	302 to `view.asp?id=`, or form with errors
`GET`	`/view.asp?id=<id>`	Read one record	Details HTML or 404
`GET`	`/edit.asp?id=<id>`	Show edit form	Pre-filled form HTML or 404
`POST`	`/edit.asp?id=<id>`	Update a record	302 to details
`POST`	`/delete.asp`	Delete a record (id + CSRF token in body)	302 to dashboard; GET returns 405

Rules every POST page follows

CSRF token

Generate a random token once per session, store it in `Session("csrf")`, put it in a hidden field, and reject any POST whose value does not match (status 403).

Validation

Check lengths, allowed status, date format (`yyyy-mm-dd` via `IsDate`), and notes length on the server. Show messages and re-fill the form with encoded values. The CHECK constraints are the last line of defence.

POST/Redirect/GET

After a successful insert, read `SCOPE_IDENTITY()`, set a flash message in `Session`, and `Response.Redirect "view.asp?id=" & newId`.

Target project structure

```
placement_tracker\
├── default.asp
├── new.asp
├── view.asp
├── edit.asp
├── delete.asp
├── 404.asp
├── web.config            (default document, custom 404, deny /db and /inc)
├── inc\
│   ├── db.asp            (connection string, OpenDb)
│   ├── helpers.asp       (H() = Server.HTMLEncode, NotFound, CSRF, flash)
│   ├── validate.asp      (shared form validation)
│   ├── form.asp          (shared create/edit form markup)
│   ├── header.asp
│   └── footer.asp
├── static\
│   └── styles.css
├── db\
│   ├── schema.sql
│   ├── seed.sql
│   └── verify.sql
├── tests\
│   ├── checklist.md
│   └── smoke.ps1
└── README.md
```

---

05 · BUILD SEQUENCE

Ten hours, ten visible wins

Do not start the next block until its deliverable works in the browser.

HOUR 1

Start the web server

IIS, sites, app pools, ASP pages, requests, and responses

Enable ASP in Windows Features, create the IIS site and app pool, create `default.asp` that writes a heading, and turn on browser error messages for development.

Visible win: Opening `http://localhost:8080/` shows "Placement Tracker" in the browser.

HOUR 2

Build the dashboard

Includes, static files, links, semantic HTML, and CSS

Create `inc/header.asp` and `inc/footer.asp` and include them on the dashboard. Render three temporary records from a VBScript array into an HTML table with a `For` loop, printing every value through `Server.HTMLEncode`.

Visible win: The browser shows a styled table generated from VBScript data.

HOUR 3

Add the database

Tables, types, constraints, primary keys, ADO connections, queries, and persistence

Install SQL Server Express and SSMS, run `schema.sql` and `seed.sql`, grant the app pool login access, write `inc/db.asp`, and replace the temporary array with an ADO `SELECT`.

Visible win: Records remain on the dashboard after `iisreset`.

HOUR 4

Create applications

Forms, POST requests, Request.Form, validation, parameterized INSERT, and redirects

Build `new.asp`: on GET show the form with a CSRF token; on POST validate, insert with `ADODB.Command` parameters, fetch `SCOPE_IDENTITY()`, and follow the Post/Redirect/Get pattern. Display errors for missing or invalid values.

Visible win: Submit the form and see the new application on the dashboard.

HOUR 5

Read one application

Query strings, ID checks, single-record queries, and 404 responses

Link every dashboard row to `view.asp?id=`. Reject non-numeric IDs, add `404.asp`, and set `Response.Status = "404 Not Found"` for an ID that does not exist.

Visible win: Each row opens the correct details page; an unknown ID shows a useful 404 page with a real 404 status.

HOUR 6

Update applications

Pre-filled forms, shared includes, validation, and parameterized UPDATE

Move the form markup into `inc/form.asp` and reuse it in `edit.asp`. Load existing values, validate changes, run a parameterized `UPDATE`, and redirect to the details page.

Visible win: Change an application from Applied to Interview and see it everywhere.

HOUR 7

Delete safely

Destructive actions, POST semantics, CSRF tokens, confirmation, and redirects

Add a delete form on the details page that posts the id and CSRF token to `delete.asp`, with `onsubmit="return confirm(...)"`. `delete.asp` returns 405 for anything other than POST.

Visible win: Delete a chosen record and return to the updated dashboard; visiting `delete.asp` directly deletes nothing.

HOUR 8

Make it robust

Edge cases, validation boundaries, error messages, encoding, and empty states

Test blank fields, invalid statuses, long notes, bad dates, missing records, an empty table, and input like `<script>` and `' OR 1=1 --`. Preserve entered values when validation fails. Set `Response.CodePage = 65001` and UTF-8 charset so non-English names survive.

Visible win: Complete the failure checklist without an ASP error page or a silent bad save.

HOUR 9

Add verification

Manual checklists, HTTP smoke tests, status codes, and SQL verification

Write `tests/checklist.md` covering every CRUD path and failure case. Write `smoke.ps1` to check status codes, and `verify.sql` to confirm what is in the table. Run them against a separate `PlacementTracker_Test` database.

```
# tests/smoke.ps1
$base = "http://localhost:8080"
function Code($url) { try { (Invoke-WebRequest $url -UseBasicParsing).StatusCode }
                      catch { [int]$_.Exception.Response.StatusCode } }
@{ "/" = 200; "/new.asp" = 200; "/view.asp?id=999999" = 404;
   "/view.asp?id=abc" = 404; "/delete.asp" = 405 }.GetEnumerator() | ForEach-Object {
  $got = Code ($base + $_.Key)
  "{0,-25} expected {1} got {2} {3}" -f $_.Key, $_.Value, $got, ($(if ($got -eq $_.Value)
}
```

```
-- db/verify.sql
SELECT COUNT(*) AS total FROM dbo.Applications;
SELECT status, COUNT(*) FROM dbo.Applications GROUP BY status;
SELECT * FROM dbo.Applications WHERE status <> N'Wishlist' AND applied_on IS NULL;  -- mu
SELECT TOP 5 * FROM dbo.Applications ORDER BY created_at DESC;
```

Visible win: The smoke script prints PASS for every line and the checklist is fully ticked.

HOUR 10

Polish, document, and demonstrate

Refactoring, accessibility, responsive design, setup instructions, and explanation

Remove duplication into includes, improve labels and navigation, check the layout on a narrow screen, switch off browser error details, write the README (Windows Features, SQL setup, permissions, site creation), and rehearse the complete user journey.

Visible win: A fresh user can set up the project on a clean Windows machine and complete the full CRUD journey from the README.

06 · CONCEPT CHECKS

Be able to explain the application

Browser and HTTP

- What happens after entering the local URL?
- Why do create and delete use POST?
- What is a redirect, and why use one after saving?
- What does a 404 response mean, and how does `Response.Status` set it?

Classic ASP and IIS

- How does IIS decide which `.asp` file runs?
- What is the difference between `Request.QueryString` and `Request.Form`?
- Why share a header and footer through includes?
- Why must every value go through `Server.HTMLEncode`?

Database and ADO

- What does one row in `dbo.Applications` represent?
- Why does every record need a primary key?
- Why are parameters safer than concatenated SQL?
- What survives an IIS restart, and why?

Full request flow

- Trace form submission from browser to database.
- Trace dashboard data from database to HTML.
- Where is input validated, and what do the CHECK constraints add?
- Which files change for a new field?

Practical understanding check

Add a `contact_person` field. Before changing anything, name every file that must change and why (schema, validation, shared form, view, dashboard if shown, checklist, verify script). Then implement it with an `ALTER TABLE` script and update the tests.

07 · COMPLETION

Definition of done

Behavior

- Create works with validation.
- Dashboard and details show saved data.
- Edit persists every field.
- Delete requires a POST with a valid CSRF token.
- Unknown IDs return a custom 404 with a 404 status.
- Data survives an IIS restart.

Quality

- Smoke script passes and the checklist is complete.
- Pages share one header and footer include.
- No SQL is built by concatenating user input.
- All output is HTML-encoded; forms have visible labels.
- Navigation works without browser Back.
- README setup works on a clean machine; the request flow can be explained.

---

08 · NEXT STEP

Move toward a modern stack

After this application works, keep the SQL Server database and add a small JSON endpoint beside the pages (for example `api/list.asp` returning JSON). Then rebuild the same routes in ASP.NET Core or FastAPI against the same tables.

Learned here	Later in ASP.NET Core / FastAPI
One `.asp` file per route	Controller actions or path-operation decorators
Request.Form and manual validation	Model binding with validation attributes or Pydantic models
Includes and Server.HTMLEncode	Razor or Jinja layouts with automatic encoding
ADO commands with parameters	Entity Framework, Dapper, or SQLAlchemy over the same schema
Session-based CSRF token	Built-in anti-forgery middleware
Checklist and PowerShell smoke tests	xUnit / pytest with a test client
HTTP methods and status codes	The same web standards